

Data Structures

Lecture 2

Algorithms

References:

Introduction to Algorithms, Data Structures and Formal Languages.

Dinneen, M.J., Gimel'farb, G.L. and Wilson, M.C., 2009.



University of Kufa
Faculty of Computer Science
and Mathematics

Dr. Munqath Al-Attar

Information Technology Research and
Development Center ITRDC

Email:

munqith.alattar@uokufa.edu.iq

Website:

<http://staff.uokufa.edu.iq/profile.html?munqith.alattar>

Topics

- **Algorithms**
 - **Introduction to Algorithms**
 - **Efficiency of Algorithms (Complexity)**



Algorithms

Introduction to Algorithms

An algorithm is a list of rules that specify successive steps to solve a problem.

A computer program is a sequence of computer instructions implementing the algorithm.

- ❖ For example, a detailed procedure for making a cake could be thought of as an algorithm.

Sorting a database, finding an entry in a database, finding a pattern in a text document.

To compare different algorithms for the same problem:

- **Correctness** whether the algorithm is correct (does it solve the problem for all legal inputs),
- **Complexity** how efficient it is (how much time, memory storage, or other resources it uses).



Algorithms

Efficiency of Algorithms (Complexity)

Two properties of an algorithm

- **The input data size n** , to be processed when solving a given problem, n is:
 - the number of items to sort (in sorting applications),
 - number of picture elements (pixels) in image processing,
 - length of a character string in text processing, and so on.
- **The number of elementary operations** taken by a particular algorithm.

It is a function $f(n)$ of the input data size n . Such as:

- Arithmetic operations (negation, addition, subtraction, multiplication, division, ...),
- Boolean operations,
- binary comparisons ("equals", "less than", or "greater than"), and so on.

The running time of an algorithm is $c f(n)$ where c is a constant factor.



Algorithms

Efficiency of Algorithms (Complexity)

The running time (or computing time) of an algorithm is the number of its elementary operations.

- ❖ The memory space required for running an algorithm depends on how many variables (input and output data) are involved at each computing step.

Mostly, time requirements are used to measure effectiveness of algorithms and programs.

Algorithms

Efficiency of Algorithms (Complexity)

Example 1.3 (Sum of elements of an array).

a is an array of integers and we need to find

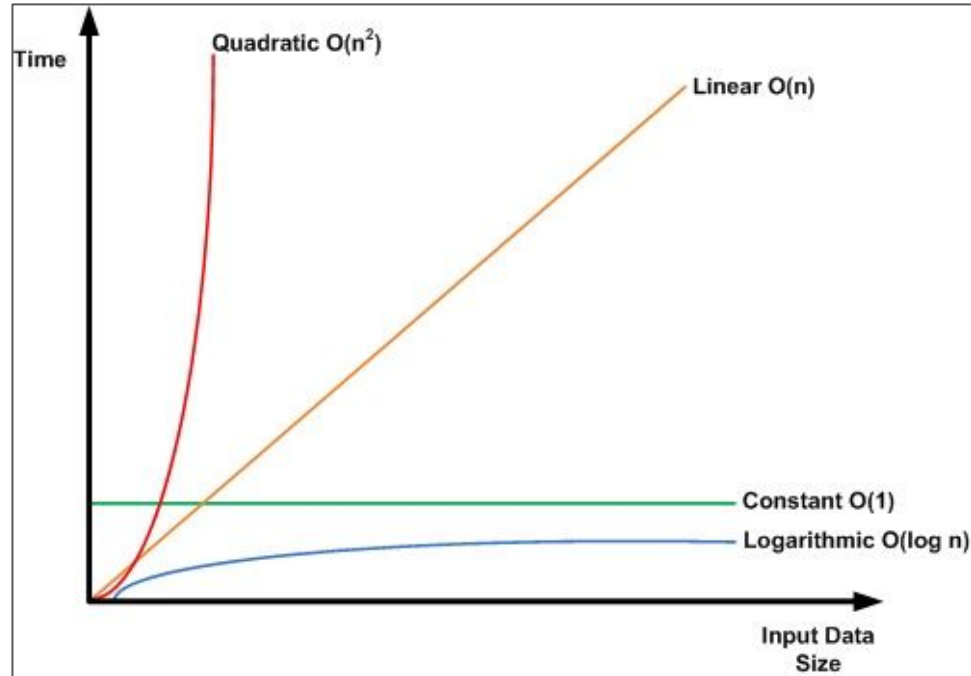
$$\text{the sum } s = \sum_{i=0}^{n-1} a[i]$$

Repeat the same elementary operations n times .

Thus, running time $T(n)$ is linear in n : $T(n) = cn$.

Such algorithms are called **linear algorithms**.

linear algorithms: the running time increases at most linearly with the size of the input.



Algorithms

Efficiency of Algorithms (Complexity)

Example 1.3 (Sum of elements of an array).

a is an array of integers and we need to find

$$\text{the sum } s = \sum_{i=0}^{n-1} a[i]$$

The running time $T(n) = cn$.

The change in running time is just the same as the change in the data size:

- $T(10) = 10T(1)$,
- $T(1000) = 1000T(1)$,
- $T(1000) = 10T(100)$,
- ...

```
algorithm linearSum
```

```
    Input: array  $a[0..n-1]$ 
```

```
begin
```

```
     $s \leftarrow 0$ 
```

```
    for  $i \leftarrow 0$  step  $i \leftarrow i + 1$  until  $n - 1$  do
```

```
         $s \leftarrow s + a[i]$ 
```

```
    end for
```

```
    return  $s$ 
```

```
end
```

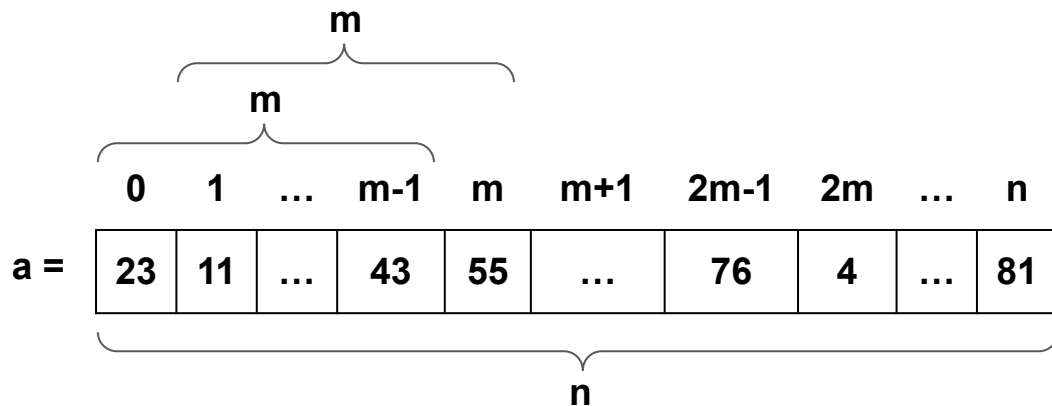
Figure 1.1: Linear-time algorithm to sum an array.

Algorithms

Efficiency of Algorithms (Complexity)

Example 1.4 (Sums of subarrays). The problem is to compute, for each subarray of size m (a subarray is $a[j..j+m-1]$) in an array a of size n .

The total number of these subarrays is $n-m+1$. So that the running time is $m(n-m+1)$.



Algorithms

Efficiency of Algorithms (Complexity)

If m is fixed, the time depends still linearly on n .

If m is growing with n as a fraction of n , such as $m = n/2$, then $T(n) = c (n/2)(n/2 + 1) = 0.25cn^2 + 0.5cn$.

n	$T(n)$	$0.25n^2$	$0.5n$	
			value	% of quadratic term
10	30	25	5	20.0
50	650	625	25	4.0
100	2550	2500	50	2.0
500	62750	62500	250	0.4
1000	250500	250000	500	0.2
5000	6252500	6250000	2500	0.04

```
algorithm slowSums
  Input: array  $a[0..2m-1]$ 
begin
  array  $s[0..m]$ 
  for  $i \leftarrow 0$  to  $m$  do
     $s[i] \leftarrow 0$ 
    for  $j \leftarrow 0$  to  $m-1$  do
       $s[i] \leftarrow s[i] + a[i+j]$ 
    end for
  end for
  return  $s$ 
end
```

Figure 1.2: Quadratic time algorithm to compute sums of an array.

Algorithms

Efficiency of Algorithms (Complexity)

For large n , the running time get close to n^2 . Such algorithms called **quadratic algorithms**.

Quadratic algorithms: the runtime is the square of the size of the input. If $T(n) \approx cn^2$ then $T(10) \approx 100T(1)$, or $T(100) \approx 10000T(1)$, or $T(100) \approx 100T(10)$.

n	$T(n)$	$0.25n^2$	$0.5n$	
			value	% of quadratic term
10	30	25	5	20.0
50	650	625	25	4.0
100	2550	2500	50	2.0
500	62750	62500	250	0.4
1000	250500	250000	500	0.2
5000	6252500	6250000	2500	0.04

```
algorithm slowSums
    Input: array  $a[0..2m-1]$ 
begin
    array  $s[0..m]$ 
    for  $i \leftarrow 0$  to  $m$  do
         $s[i] \leftarrow 0$ 
        for  $j \leftarrow 0$  to  $m-1$  do
             $s[i] \leftarrow s[i] + a[i+j]$ 
        end for
    end for
    return  $s$ 
end
```

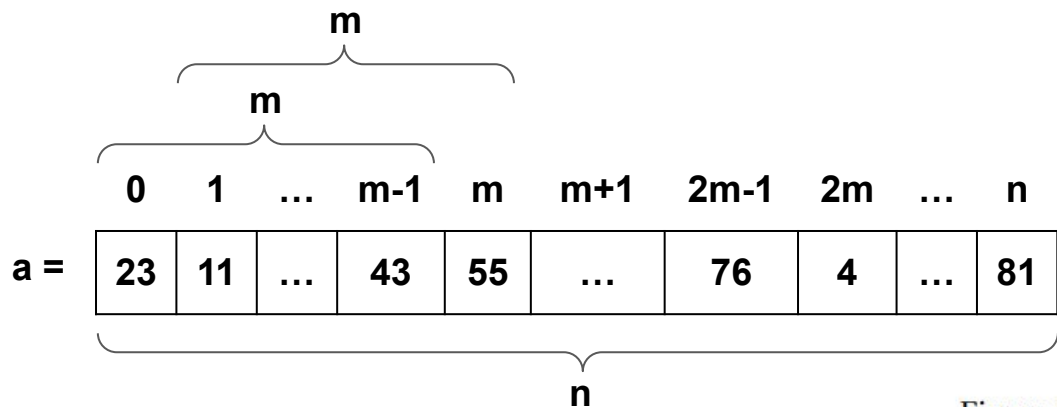
Figure 1.2: Quadratic time algorithm to compute sums of an array.

Algorithms

Efficiency of Algorithms (Complexity)

The repeated computations in the innermost loop are unnecessary. Two successive sums $s[i]$ and $s[i-1]$ differ only by two elements: $s[i] = s[i-1] + a[i+m-1] - a[i-1]$. Thus we need not repeatedly add m items together after getting the very first sum $s[0]$.

$$T(n) = c(m + 2(n - m)) = c(2n - m)$$



```
algorithm fastSums
  Input: array  $a[0..2m-1]$ 
begin
  array  $s[0..m]$ 
   $s[0] \leftarrow 0$ 
  for  $j \leftarrow 0$  to  $m-1$  do
     $s[0] \leftarrow s[0] + a[j]$ 
  end for
  for  $i \leftarrow 1$  to  $m$  do
     $s[i] \leftarrow s[i-1] + a[i+m-1] - a[i-1]$ 
  end for
  return  $s$ ;
end
```

Figure 1.3: Linear-time algorithm to compute sums of an array.

Algorithms

Exercises

Exercise 1.1.1. A quadratic algorithm with processing time $T(n) = cn^2$ uses 500 elementary operations for processing 10 data items. How many will it use for processing 1000 data items?

Exercise 1.1.2. Algorithms A and B use exactly $T_A(n) = c_A n \log_2 n$ and $T_B(n) = c_B n^2$ elementary operations, respectively, for a problem of size n . Find the fastest algorithm for processing $n = 2^{20}$ data items if A and B spend 10 and 1 operations, respectively, to process $2^{10} \equiv 1024$ items.